გაკვეთილი 4 Mod 7 (7.1)

კი, გავაგრძელოთ ზუსტად წინა სამი გაკვეთილის სტილში. მეოთხე გაკვეთილი იქნება **Branch-ები და Git-ის სრული სამუშაო პროცესი** — ანუ ბოლოს უკვე პირველი ოთხი გაკვეთილის ცოდნასაც გავაერთიანებთ.

გაკვეთილი 4 — Git Branch-ები და სრული Git Workflow

თემა: Branch-ის შექმნა, branch-ებს შორის გადასვლა, ცვლილებების დამოუკიდებლად გაკეთება და GitHub-ზე ატვირთვა
ხანგრძლივობა: 40 წუთი

წინა გაკვეთილებიდან მოსწავლეებმა უკვე იციან: `git init`, `git status`, `git add`, `git commit`, `git log`, Local/Remote Repository, `origin`, `git push` და GitHub-ზე პროექტის ატვირთვა.

გაკვეთილის მიზანი: მოსწავლემ გაიგოს, რა არის Branch, რატომ ვიყენებთ მას, შეძლოს ახალი Branch-ის შექმნა, Branch-ებს შორის გადასვლა და ცვლილებების GitHub-ზე გაგზავნა.

---

1. წინა გაკვეთილის მოკლე გამეორება

უკვე ვიცით Git-ის ძირითადი სამუშაო პროცესი:

```
კოდის შეცვლა
      ↓
git status
      ↓
git add .
      ↓
git commit -m "message"
      ↓
Local Repository
      ↓
git push
      ↓
GitHub
```

მაგალითად:

```
git status
git add .
git commit -m "Add contact section"
git push
```

მაგრამ ახლა წარმოვიდგინოთ სხვა სიტუაცია.

ჩვენი საიტი უკვე მუშაობს და `main`-ში გვაქვს:

```
school-website/
│
├── index.html
├── style.css
└── script.js
```

გვინდა დავამატოთ ახალი ფუნქცია — მაგალითად **Gallery**.

თუ პირდაპირ ძირითად პროექტში დავიწყებთ მუშაობას და რაღაც გავაფუჭეთ, შეიძლება მოქმედი ვერსიაც დაზიანდეს.

სწორედ აქ გვჭირდება **Branch**.

---

2. რა არის Branch?

Branch ნიშნავს პროექტის განვითარების ცალკე მიმართულებას.

მარტივად რომ ვთქვათ:

> Branch გვაძლევს საშუალებას პროექტის ცალკე ვერსიაზე ვიმუშაოთ ისე, რომ ძირითადი სამუშაო ვერსია არ შევცვალოთ.

ძირითადი Branch ჩვეულებრივ არის:

```
main
```

მაგალითად, გვაქვს:

```
main
 │
 ├── Commit 1
```

```
  ├── Commit 2
  └── Commit 3
```

ახლა გვინდა Gallery-ის დამატება.

შეგვიძლია შევქმნათ:

```
gallery
```

Branch.

სქემა დაახლოებით ასეთი გახდება:

```
                ┌── gallery
                │
Commit 1 — Commit 2 — Commit 3
                │
                └── main
```

შემდეგ `gallery` Branch-ზე შეგვიძლია დამოუკიდებლად ვიმუშაოთ.

---

3. რატომ ვიყენებთ Branch-ებს?

ვთქვათ, ჩვენი საიტის ძირითადი ვერსია მუშაობს.

```
main
↓
მუშა ვებგვერდი
```

გვინდა ახალი Navigation Menu-ის დამატება.

შეგვიძლია შევქმნათ:

```
navigation
```

და ცვლილებები იქ გავაკეთოთ.

თუ ყველაფერი კარგად მუშაობს, მოგვიანებით ამ ცვლილებებს `main`-თან გავაერთიანებთ.

თუ ცუდად მუშაობს, `main` მაინც უსაფრთხოდ რჩება.

ამიტომ Branch განსაკუთრებით სასარგებლოა:

- ახალი ფუნქციის დამატებისას;
- ექსპერიმენტების დროს;
- შეცდომების გასწორებისას;
- გუნდური მუშაობისას;
- მაშინ, როდესაც არ გვინდა ძირითადი ვერსიის პირდაპირ შეცვლა.

---

4. რეალური მაგალითი

ვთქვათ, ვაკეთებთ სკოლის საიტს.

ძირითადი Branch:

```
main
```

აქ არის საიტის მოქმედი ვერსია.

ერთმა ადამიანმა უნდა დაამატოს Gallery.

ქმნის:

```
gallery
```

მეორემ უნდა დაამატოს Contact გვერდი.

ქმნის:

```
contact
```

მესამე მუშაობს Navigation-ზე:

```
navigation
```

შედეგად:

```
              gallery
             /
main ———————●
             \
```

```
                contact

        + navigation
```

ყველას შეუძლია სხვადასხვა ცვლილებაზე მუშაობა ისე, რომ თავიდან ერთმანეთის კოდს ხელი არ შეუშალონ.

---

5. როგორ ვნახოთ Branch-ები?

ამისთვის გამოიყენება:

```
git branch
```

მაგალითად:

```
* main
```

ვარსკვლავი:

```
*
```

გვიჩვენებს, რომელ Branch-ზე ვიმყოფებით ამჟამად.

თუ გვაქვს:

```
  gallery
* main
```

ეს ნიშნავს:

- გვაქვს `gallery`;
- გვაქვს `main`;
- ამჟამად ვართ `main`-ზე.

---

6. ახალი Branch-ის შექმნა

ახალი Branch-ის შესაქმნელად შეგვიძლია გამოვიყენოთ:

```
git branch gallery
```

ახლა:

```
git branch
```

შეიძლება გვაჩვენოს:

```
  gallery
* main
```

ეს ნიშნავს, რომ `gallery` შევქმენით.

მაგრამ ჯერ ისევ `main`-ზე ვიმყოფებით.

ეს ძალიან მნიშვნელოვანია:

> `git branch gallery` ქმნის Branch-ს, მაგრამ ავტომატურად მასზე არ გადაგვიყვანს.

---

7. Branch-ზე გადასვლა — `git switch`

Branch-ის შესაცვლელად შეგვიძლია გამოვიყენოთ:

```
git switch gallery
```

ახლა:

```
git branch
```

გვაჩვენებს:

```
* gallery
  main
```

ვარსკვლავი უკვე `gallery`-ს უწერია.

ანუ ახლა ყველა ახალი Commit გაკეთდება `gallery` Branch-ზე.

---

8. `git checkout`

ძველ პროექტებში და ბევრ სასწავლო მასალაში Branch-ზე გადასასვლელად შეგხვდებათ:

```
git checkout gallery
```

ისიც Branch-ზე გადაგვიყვანს.

მაგრამ თანამედროვე Git-ში Branch-ებს შორის გადასვლისთვის უფრო გასაგები ბრძანებაა:

```
git switch gallery
```

ამიტომ გაკვეთილზე ძირითადად გამოვიყენოთ `git switch`, მაგრამ მოსწავლეებმა `checkout`-იც უნდა იცნონ, რადგან ინტერნეტში ხშირად შეხვდებათ.

---

9. Branch-ის შექმნა და მასზე პირდაპირ გადასვლა

შეგვიძლია ორი ბრძანება გამოვიყენოთ:

```
git branch gallery
git switch gallery
```

მაგრამ ამის გაკეთება ერთდროულადაც შეგვიძლია:

```
git switch -c gallery
```

`-c` ნიშნავს ახალი Branch-ის შექმნას.

ანუ:

```
git switch -c gallery
```

აკეთებს ორივეს:

```
ქმნის gallery Branch-ს
        +
გადაგვიყვანს gallery Branch-ზე
```

ეს ძალიან მოსახერხებელი ბრძანებაა.

10. პრაქტიკული მაგალითი

გვაქვს პროექტი:

```
school-website/
│
├── index.html
└── style.css
```

ვიმყოფებით:

```
main
```

ვამოწმებთ:

```
git branch
```

შემდეგ ვქმნით:

```
git switch -c gallery
```

ვამოწმებთ:

```
git branch
```

ახლა უნდა გვქონდეს:

```
* gallery
  main
```

ანუ უკვე `gallery` Branch-ზე ვმუშაობთ.

---

11. ცვლილების გაკეთება Branch-ზე

`index.html` -ში დავამატოთ:

```
<section>
    <h2>Gallery</h2>
    <p>Our school gallery</p>
</section>
```

ახლა:

```
git status
```

დავინახავთ, რომ `index.html` შეიცვალა.

შემდეგ:

```
git add .
```

და:

```
git commit -m "Add gallery section"
```

ახლა ეს Commit გაკეთებულია **gallery Branch-ზე**.

---

12. რა მოხდა?

თავიდან გვქონდა:

```
main
 |
 ● Create homepage
 |
 ● Add styles
```

შევქმენით `gallery` და იქ ახალი Commit გავაკეთეთ.

ახლა დაახლოებით გვაქვს:

```
main
 |
 ● Create homepage
 |
 ● Add styles
```

```
   |
   └──────────────● Add gallery section
                  ↑
                gallery
```

`gallery` შეიცავს ახალ ცვლილებას.

`main` კი ჯერ ისევ ძველ მდგომარეობაშია.

---

13. დავბრუნდეთ `main` -ზე

ვწერთ:

```
git switch main
```

ახლა:

```
git branch
```

გვაჩვენებს:

```
  gallery
* main
```

თუ `index.html` -ს შევხედავთ, Gallery-ის ცვლილება შეიძლება აღარ იყოს `main` -ში, რადგან ის `gallery` Branch-ზე გავაკეთეთ.

ეს Branch-ის იდეის ძალიან კარგი პრაქტიკული დემონსტრაციაა.

---

14. ისევ Gallery-ზე დაბრუნება

```
git switch gallery
```

Gallery-ის კოდი ისევ შესაბამის Branch-ზე გვექნება.

ანუ Branch-ებს შორის გადასვლისას Git პროექტის ფაილებს შესაბამისი Branch-ის მდგომარეობას უსადაგებს.

ეს კარგად აჩვენებს, რომ:

> სხვადასხვა Branch-ზე პროექტის სხვადასხვა განვითარების მდგომარეობა შეიძლება გვქონდეს.

---

15. რა არის Merge?

ახლა ვთქვათ Gallery დავასრულეთ და ყველაფერი კარგად მუშაობს.

გვინდა Gallery-ის ცვლილება ძირითად `main` Branch-შიც გადავიტანოთ.

ამისთვის გამოიყენება:

```
git merge
```

Merge ნიშნავს Branch-ების ცვლილებების გაერთიანებას.

---

16. `gallery` Branch-ის `main` -ში გაერთიანება

ჯერ უნდა გადავიდეთ იმ Branch-ზე, **სადაც ცვლილების დამატება გვინდა**.

ჩვენ შემთხვევაში:

```
git switch main
```

შემდეგ:

```
git merge gallery
```

ამით ვეუბნებით Git-ს:

> `gallery` Branch-ის ცვლილებები გააერთიანე ჩემს მიმდინარე `main` Branch-ში.

---

17. Merge-ის მარტივი სქემა

Merge-მდე:

```
main
  |
  ●──────────
   \
    ● Add gallery
    ↑
   gallery
```

Merge-ის შემდეგ `main`-შიც იქნება Gallery-ის ცვლილება:

```
main
 |
 ●
 |
 ● Add gallery
 ↑
main
```

ანუ Gallery უკვე ძირითადი პროექტის ნაწილია.

---

18. Merge-ის ყველაზე მნიშვნელოვანი წესი

თუ გვინდა:

```
gallery → main
```

ჯერ უნდა ვიყოთ:

```
git switch main
```

და შემდეგ:

```
git merge gallery
```

შეგვიძლია ასე დავიმახსოვროთ:

> ჯერ გადავდივართ იმ Branch-ზე, **რომელშიც გვინდა ცვლილებების მიღება**, შემდეგ ვაკეთებთ `merge`-ს.

---

19. Branch-ის GitHub-ზე ატვირთვა

თუ `gallery` Branch-ის GitHub-ზე ატვირთვაც გვინდა:

```
git switch gallery
```

შემდეგ:

```
git push -u origin gallery
```

ახლა GitHub-ზე გვექნება არა მხოლოდ:

```
main
```

არამედ:

```
gallery
```

Branch-იც.

შემდეგ ამ Branch-ზე ახალი Commit-ების გაკეთებისას ხშირად საკმარისი იქნება:

```
git push
```

---

20. Merge-ის შემდეგ `main` -ის GitHub-ზე ატვირთვა

ვთქვათ გავაკეთეთ:

```
git switch main
git merge gallery
```

ლოკალურ `main` -ში Gallery უკვე არის.

მაგრამ GitHub-ზე ცვლილების გასაგზავნად მაინც გვჭირდება:

```
git push
```

ანუ:

```
gallery
   ↓
git merge
   ↓
Local main
   ↓
git push
   ↓
GitHub main
```

---

21. სრული პრაქტიკული მაგალითი

გვაქვს მოქმედი ვებგვერდი და გვინდა **About** სექციის დამატება.

1. შევამოწმოთ მიმდინარე Branch

```
git branch
```

2. შევქმნათ ახალი Branch და გადავიდეთ მასზე

```
git switch -c about-section
```

3. შევამოწმოთ

```
git branch
```

უნდა გამოჩნდეს:

```
* about-section
  main
```

4. შევცვალოთ `index.html`

```
<section>
    <h2>About Us</h2>
    <p>Welcome to our school.</p>
</section>
```

5. შევამოწმოთ

```
git status
```

6. მოვამზადოთ ცვლილება

```
git add .
```

7. შევქმნათ Commit

```
git commit -m "Add about section"
```

8. ავტვირთოთ Branch GitHub-ზე

```
git push -u origin about-section
```

9. დავბრუნდეთ `main`-ზე

```
git switch main
```

10. გავაერთიანოთ

```
git merge about-section
```

11. განახლებული `main` ავტვირთოთ GitHub-ზე

```
git push
```

---

22. სრული Workflow

ახლა უკვე შეგვიძლია Git-ის ოთხივე გაკვეთილი ერთ სამუშაო პროცესში გავაერთიანოთ.

პირველად:

```
git init
```

Remote-ის დამატება:

```
git remote add origin REPOSITORY_URL
git branch -M main
git push -u origin main
```

ახალ ფუნქციაზე მუშაობისას:

```
git switch -c new-feature
```

ვცვლით კოდს.

შემდეგ:

```
git status
git add .
git commit -m "Add new feature"
```

სურვილის შემთხვევაში Branch-ს ვაგზავნით GitHub-ზე:

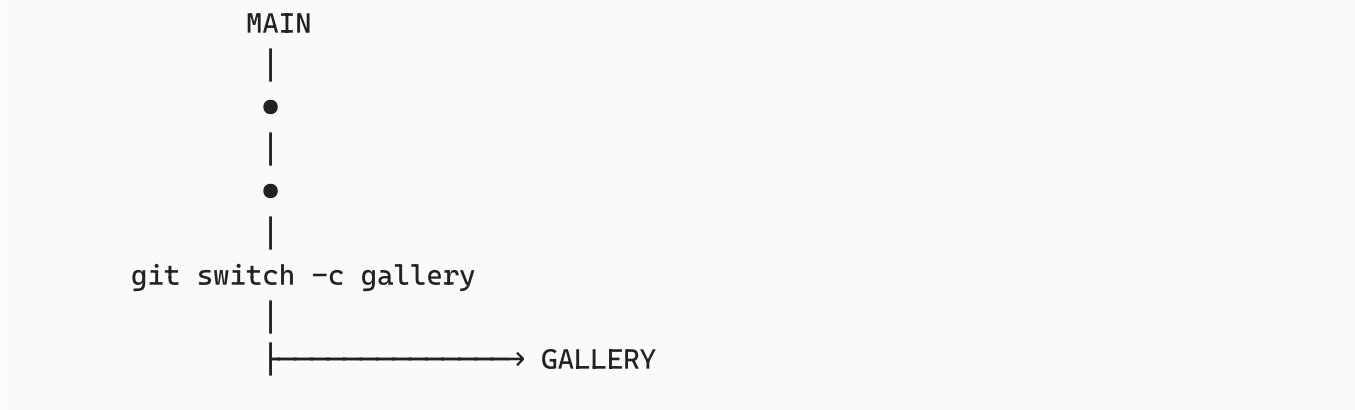
```
git push -u origin new-feature
```

როდესაც ფუნქცია მზადაა:

```
git switch main
git merge new-feature
git push
```

---

23. Git Workflow-ის ვიზუალური სქემა



```
          |                    |
          |                ვცვლით კოდს
          |                    ↓
          |                git add .
          |                    ↓
          |                git commit
          |                    ↓
          |                   ●
          |                   |
          |              git push
          |
          |
    git switch main
          ↓
git merge gallery
          ↓
         MAIN
          ↓
       git push
          ↓
        GitHub
```

---

24. Merge Conflict — რა არის?

ამ გაკვეთილზე Conflict-ის დეტალურად გადაწყვეტა აუცილებელი არ არის, მაგრამ მოსწავლემ უნდა იცოდეს, რომ ასეთი რამ არსებობს.

ვთქვათ `main` Branch-ზე ერთი ადამიანი შეცვლის:

```
<h1>Welcome to School</h1>
```

ხოლო სხვა Branch-ზე იგივე ხაზი შეიცვლება:

```
<h1>Our School Website</h1>
```

Merge-ის დროს Git-ს შეიძლება გაუჭირდეს გადაწყვიტოს, რომელი ვერსია უნდა დარჩეს.

ამას ეწოდება:

Merge Conflict

მარტივად:

> Merge Conflict წარმოიქმნება მაშინ, როდესაც Git ავტომატურად ვერ წყვეტს, როგორ გააერთიანოს ერთმანეთთან წინააღმდეგობაში მყოფი ცვლილებები.

ამ შემთხვევაში პროგრამისტმა თავად უნდა გადაწყვიტოს, რომელი კოდი უნდა დარჩეს.

---

25. პრაქტიკული სავარჯიშო 1 — Branch-ის შექმნა

მოსწავლეს აქვს `student-website`.

უნდა:

1. ნახოს არსებული Branch-ები.

2. შექმნას:

```
contact-page
```

3. გადავიდეს მასზე.

ამის გაკეთება შესაძლებელია:

```
git branch
git switch -c contact-page
git branch
```

ბოლოს `*` უნდა იყოს:

```
* contact-page
  main
```

---

26. პრაქტიკული სავარჯიშო 2 — Commit ახალ Branch-ზე

`contact-page` Branch-ზე `index.html`-ში დაამატოს:

```
<section>
    <h2>Contact Us</h2>
```

```
    <p>Email: school@example.com</p>
</section>
```

შემდეგ:

```
git status
git add .
git commit -m "Add contact section"
```

და ბოლოს:

```
git log --oneline
```

---

27. პრაქტიკული სავარჯიშო 3 — Branch-ებს შორის გადასვლა

მოსწავლემ შეასრულოს:

```
git switch main
```

და ნახოს `index.html`.

შემდეგ:

```
git switch contact-page
```

და ისევ ნახოს `index.html`.

კითხვა: რატომ განსხვავდება ფაილის მდგომარეობა?

პასუხი: იმიტომ, რომ Contact-ის ცვლილება `contact-page` Branch-ზე გაკეთდა და ჯერ `main`-ში არ გაგვიერთიანებია.

---

28. პრაქტიკული სავარჯიშო 4 — Merge

ახლა Contact სექცია `main`-ში უნდა გადავიტანოთ.

მოსწავლემ დამოუკიდებლად განსაზღვროს სწორი თანმიმდევრობა.

სწორი პასუხია:

```
git switch main
git merge contact-page
```

შემდეგ:

```
git status
```

და თუ Remote უკვე დაკავშირებულია:

```
git push
```

---

29. სავარჯიშო 5 — დაალაგე სწორი თანმიმდევრობით

გვინდა ახალი `gallery` ფუნქციის შექმნა და საბოლოოდ `main`-ში გაერთიანება.

მოცემულია:

```
git merge gallery
git add .
git switch main
git commit -m "Add gallery"
git switch -c gallery
```

დაალაგეთ სწორად.

პასუხი

```
git switch -c gallery

git add .

git commit -m "Add gallery"

git switch main

git merge gallery
```

თუ GitHub-ზეც გვინდა ცვლილება:

```
git push
```

---

30. სავარჯიშო 6 — აირჩიე სწორი ბრძანება

1. არსებული Branch-ების ნახვა:

```
git branch
```

2. ახალი `login` Branch-ის შექმნა და მასზე გადასვლა:

```
git switch -c login
```

3. `main` Branch-ზე დაბრუნება:

```
git switch main
```

4. `login` Branch-ის მიმდინარე Branch-ში გაერთიანება:

```
git merge login
```

5. `login` Branch-ის პირველად GitHub-ზე გაგზავნა:

```
git push -u origin login
```

---

31. სავარჯიშო 7 — იპოვე შეცდომა

გვინდა `gallery`-ის ცვლილებები `main`-ში გადავიტანოთ.

მოსწავლემ დაწერა:

```
git switch gallery
git merge main
```

ეს რას აკეთებს?

ამ შემთხვევაში პირიქით ხდება — `main`-ის ცვლილებების `gallery`-ში გაერთიანებას ვცდილობთ.

ჩვენ გვინდოდა:

```
gallery → main
```

ამიტომ სწორი იქნება:

```
git switch main
git merge gallery
```

---

32. საბოლოო პრაქტიკული დავალება

მოსწავლემ გამოიყენოს თავისი წინა გაკვეთილების პროექტი:

```
student-website/
│
├── index.html
└── style.css
```

უნდა შეასრულოს შემდეგი:

1. შეამოწმოს, რომელ Branch-ზე იმყოფება.

2. შექმნას ახალი Branch:

```
new-feature
```

3. `new-feature`-ზე დაამატოს ახალი სექცია `index.html`-ში.

მაგალითად:

```
<section>
    <h2>Our Services</h2>
    <p>Learn more about our services.</p>
</section>
```

4. შექმნას Commit შესაბამისი Commit Message-ით.

5. `new-feature` Branch ატვირთოს GitHub-ზე.

6. დაბრუნდეს `main` Branch-ზე.

7. გააერთიანოს `new-feature` `main`-ში.

8. განახლებული `main` ატვირთოს GitHub-ზე.

საბოლოო ბრძანებები დაახლოებით ასეთი იქნება:

```
git branch

git switch -c new-feature

git status
git add .
git commit -m "Add services section"

git push -u origin new-feature

git switch main

git merge new-feature

git push
```